

CONTENTS

Experiment No.	Project Name	Page No.
Experiment – 1	Digital Temperature Monitoring System	1
Experiment – 1	Temperature Monitoring-Emergency Alert System	8
Experiment – 1	Weather Forecasting System with Extreme Temperature Indicator	12
Experiment – 2	Simple Calculator Using Arduino UNO, 4x4 Keypad and SSD1306 OLED Display	24
Experiment – 4	Temperature Data Logger Using Arduino UNO, DHT11 Sensor, DS1307 RTC Module and SSD1306 OLED Display	34
Experiment – 4	Real-Time Alarm Clock Using Arduino UNO, DS1307 RTC Module, SSD1306 OLED Display and Push Buttons	42
Experiment – 4	Alarm Based Motor ON/OFF System Using Arduino UNO, DS1307 RTC Module and SSD1306 OLED Display	54

SOFTWARE USED

- Arduino IDE
- SimulIDE

STUDENT

Ganesh Mahata

2022BB1197

GIPEDI Program at IIT Delhi - Batch 2 2026

EXPERIMENT – 1 INTERFACING WITH ARDUINO

PROJECT NAME

Digital Temperature Monitoring System

OBJECTIVE / PROBLEM STATEMENT

The main objective of this project is to design a smart temperature monitoring system using Arduino UNO and a DHT11 temperature sensor.

The system continuously measures room temperature and displays the live temperature value on an SSD1306 OLED display. According to the temperature value, the system automatically controls cooling and heating devices such as a fan and heater.

A push button is also provided to turn the complete system ON or OFF manually.

COMPONENTS USED

Component	Quantity
Arduino UNO	1
DHT11 Temperature Sensor	1
SSD1306 OLED Display	1
Red LED	1
Green LED	1
DC Fan / Motor	1
Heater / Bulb	1
Push Button	1
Jumper Wires	Required

PIN CONNECTIONS

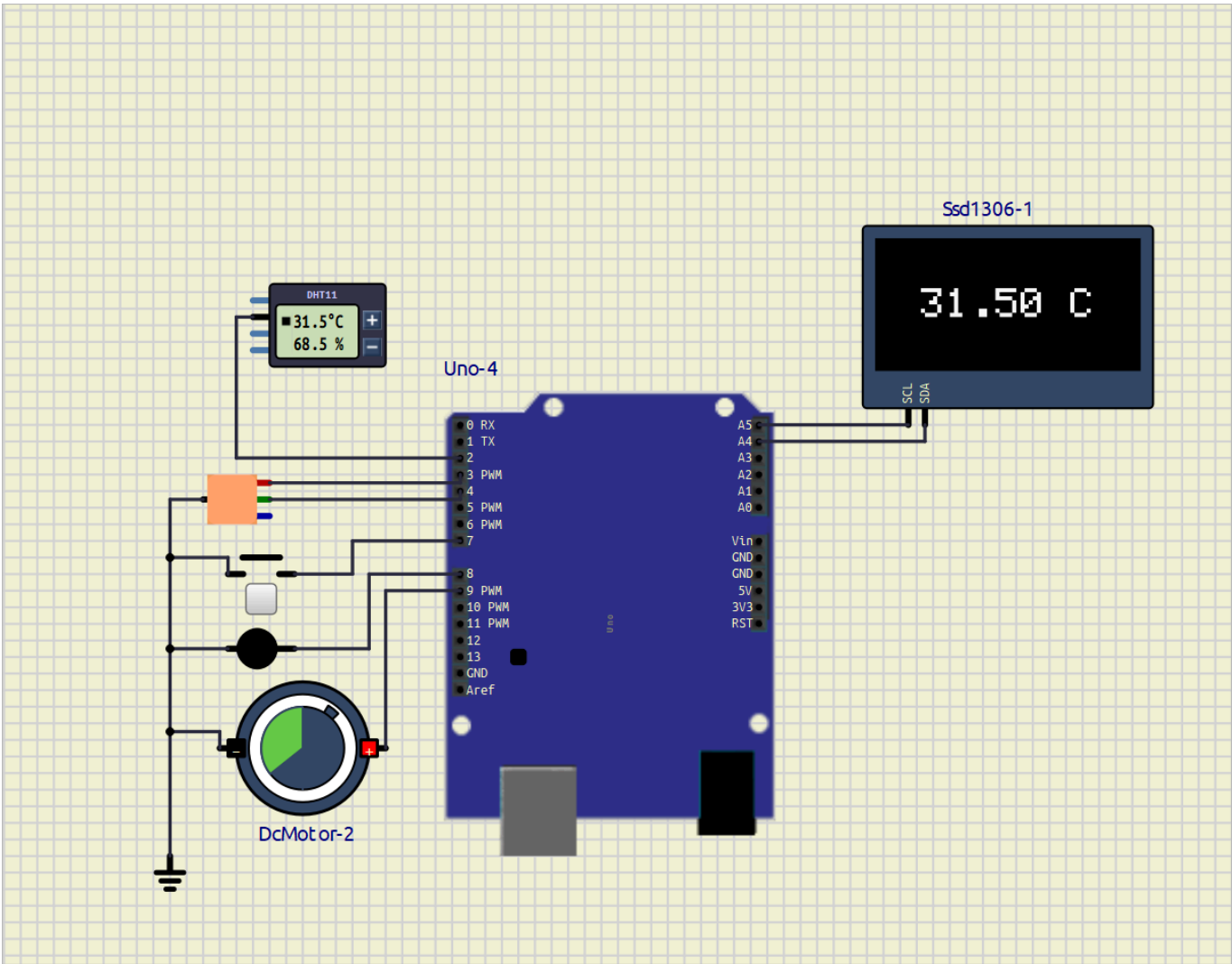
Device	Arduino Pin
DHT11 Data Pin	D2
Red LED	D3
Green LED	D4
Push Button	D7

Device	Arduino Pin
Heater	D8
Fan	D9
OLED SDA	A4
OLED SCL	A5

S SOFTWARE USED

- Arduino IDE
- SimulIDE

C CIRCUIT DIAGRAM



R REQUIRED LIBRARIES

```
#include <Wire.h>
#include <Adafruit_GFX.h>
```

```
#include <Adafruit_SSD1306.h>
#include <DHT.h>
```

MAIN ARDUINO CODE

[Code](/Exp-1/smart_temp_controler/temp_sys.ino)

```
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
#include <DHT.h>

#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64

#define DHTPIN 2
#define DHTTYPE DHT11

DHT dht(DHTPIN, DHTTYPE);

Adafruit_SSD1306 display(
  SCREEN_WIDTH,
  SCREEN_HEIGHT,
  &Wire,
  -1
);

bool systemON = false;

void setup()
{
  dht.begin();

  display.begin(SSD1306_SWITCHCAPVCC, 0x3C);

  display.clearDisplay();

  pinMode(3, OUTPUT);

  pinMode(4, OUTPUT);

  pinMode(9, OUTPUT);

  pinMode(8, OUTPUT);

  pinMode(7, INPUT_PULLUP);
}
```

```
void loop()
{
  if(digitalRead(7) == LOW)
  {
    systemON = !systemON;

    delay(300);
  }

  if(systemON)
  {
    float temp = dht.readTemperature();

    display.clearDisplay();

    display.setTextSize(2);

    display.setTextColor(WHITE);

    String text = String(temp) + " C";

    int x = (128 - (text.length() * 12)) / 2;

    int y = (64 - 16) / 2;

    display.setCursor(x, y);

    display.print(text);

    display.display();

    if(temp >= 28)
    {
      digitalWrite(3, HIGH);

      digitalWrite(4, LOW);

      digitalWrite(9, HIGH);

      digitalWrite(8, LOW);
    }
    else
    {
      digitalWrite(3, LOW);

      digitalWrite(4, HIGH);

      digitalWrite(9, LOW);

      digitalWrite(8, HIGH);
    }
  }
}
```

else

```
{  
    digitalWrite(3, LOW);  
  
    digitalWrite(4, LOW);  
  
    digitalWrite(9, LOW);  
  
    digitalWrite(8, LOW);  
  
    display.clearDisplay();  
  
    display.display();  
}  
  
delay(1000);  
}
```

WORKING PRINCIPLE

1. The push button is used to turn the complete system ON or OFF.
2. When the system is ON, the DHT11 sensor continuously measures room temperature.
3. The measured temperature value is displayed on the OLED screen in real time.
4. The Arduino UNO compares the measured temperature with the predefined threshold temperature value.
5. If the temperature becomes greater than or equal to 28°C:
 - Red LED turns ON
 - Fan starts running automatically
 - Heater turns OFF
6. If the temperature becomes lower than 28°C:
 - Green LED turns ON
 - Heater turns ON automatically
 - Fan turns OFF
7. The OLED display continuously updates the live temperature reading.
8. When the push button is pressed again:
 - The complete system turns OFF
 - OLED display becomes blank
 - LEDs turn OFF
 - Fan and heater stop working

ADVANTAGES

- Automatic temperature control
 - Real-time temperature monitoring
 - Easy ON/OFF operation
 - Low-cost implementation
 - User-friendly system
 - Useful for smart automation
-

APPLICATIONS

- Smart room temperature control
 - Automatic cooling systems
 - Automatic heating systems
 - Industrial temperature monitoring
 - Home automation projects
 - Embedded systems learning
-

CONCLUSION

The Digital Temperature Monitoring System using Arduino UNO and DHT11 sensor was successfully implemented and tested in SimulIDE.

The system continuously monitored temperature and automatically controlled the fan and heater according to the temperature value. The OLED display successfully showed live temperature readings, and the push button provided convenient ON/OFF control of the complete system.

This project helped in understanding:

- Arduino interfacing
 - Sensor interfacing
 - OLED display handling
 - Embedded system automation
 - Temperature-based control systems
-

REFERENCES

1. Arduino UNO Documentation
<https://docs.arduino.cc/hardware/uno-rev3/>
2. DHT11 Sensor Documentation
<https://components101.com/sensors/dht11-temperature-sensor>
3. Adafruit SSD1306 Library
https://github.com/adafruit/Adafruit_SSD1306
4. Adafruit GFX Library
<https://github.com/adafruit/Adafruit-GFX-Library>

5. Arduino DHT Sensor Library

<https://docs.arduino.cc/libraries/dht-sensor-library/>

6. Arduino UNO Pinout

<https://deepbluembedded.com/arduino-uno-pinout/>

EXPERIMENT – 1 INTERFACING WITH ARDUINO

PROJECT NAME

Temperature Monitoring-Emergency Alert System

OBJECTIVE / PROBLEM STATEMENT

This project monitors temperature using a DHT11 sensor and activates a buzzer when the temperature exceeds the threshold value of 50°C. The system is designed using Arduino UNO in SimulIDE.

COMPONENTS USED

Component	Quantity
Arduino UNO	1
DHT11 Temperature Sensor	1
Buzzer	1
Jumper Wires	Required

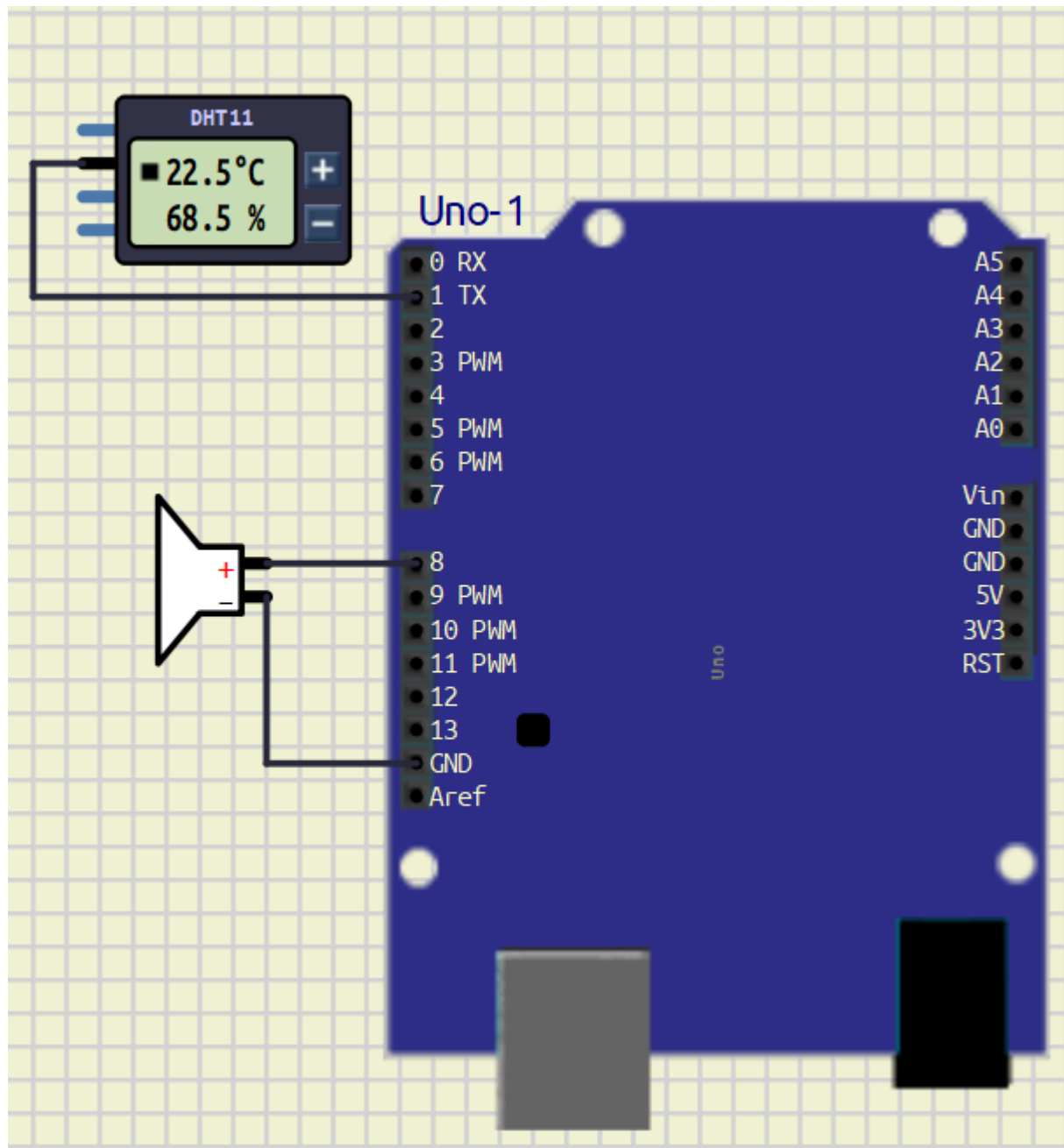
PIN CONNECTIONS

Device	Arduino Pin
DHT11 Data Pin	D1
Buzzer	D8
DHT11 VCC	5V
DHT11 GND	GND

SOFTWARE USED

- Arduino IDE
 - SimulIDE
-

Circuit diagram



REQUIRED LIBRARY

```
#include <DHT.h>
```

MAIN ARDUINO CODE

Code

```
#include <DHT.h>

#define DHTPIN 1
#define DHTTYPE DHT11
#define BUZZER 8
#define THRESHOLD 50
```

```
DHT dht(DHTPIN, DHTTYPE);

void setup()
{
    pinMode(BUZZER, OUTPUT);
    dht.begin();
}

void loop()
{
    float temp = dht.readTemperature();

    if(temp >= THRESHOLD)
    {
        digitalWrite(BUZZER, HIGH);
    }
    else
    {
        digitalWrite(BUZZER, LOW);
    }

    delay(1000);
}
```

WORKING PRINCIPLE

1. DHT11 sensor continuously reads temperature.
2. Arduino UNO compares the temperature with the threshold value.
3. If temperature is greater than or equal to 50°C:
 - Buzzer turns ON.
4. If temperature is below 50°C:
 - Buzzer remains OFF.
5. The process repeats continuously.

OUTPUT

- Buzzer remains OFF below 50°C.
- Buzzer turns ON at or above 50°C.
- System performs real-time temperature monitoring.

ADVANTAGES

- Simple and low-cost system
- Easy to implement
- Real-time monitoring
- Useful for safety applications

APPLICATIONS

- Fire warning systems
 - Temperature alert systems
 - Industrial monitoring
 - Home safety projects
 - Embedded system learning
-

CONCLUSION

The Digital Temperature Monitoring System using Arduino UNO and DHT11 sensor was successfully implemented in SimulIDE. The buzzer activated whenever the temperature crossed the threshold value of 50°C.

References

1. Arduino Project hub- With DHT11 sensor <https://projecthub.arduino.cc/arcaegecengiz/using-dht11-12f621>
2. DHT sensor Library - <https://docs.arduino.cc/libraries/dht-sensor-library/#Releases>
3. Arduino Uno board Pin Out - <https://deepbluembedded.com/arduino-uno-pinout/>

EXPERIMENT – 1 Interfacing With Arduino

PROJECT NAME

Weather Forecasting System with Extreme Temperature Indicator

OBJECTIVE / PROBLEM STATEMENT

This project monitors environmental temperature and humidity using a DHT11 sensor and predicts weather conditions based on temperature values.

The system uses RGB LED indicators to show:

- Extreme Hot Weather
- Normal Weather
- Cold Weather

A DS1307 RTC module provides real-time clock information, and an SSD1306 OLED display shows:

- Current Time
- Temperature
- Humidity
- Weather Status

A push button is also used as a master power control for turning the complete system ON and OFF.

COMPONENTS USED

Component	Quantity
Arduino UNO	1
DHT11 Sensor	1
DS1307 RTC Module	1
SSD1306 OLED Display	1
RGB LED	1
Push Button	1
Resistors (100Ω)	3
Jumper Wires	Required
Breadboard	1

PIN CONNECTIONS

Device	Arduino Pin
DHT11 Data Pin	D7
Red LED	D2
Green LED	D3
Blue LED	D5
Power Button	D8
OLED SDA	A4
OLED SCL	A5
RTC SDA	A4
RTC SCL	A5

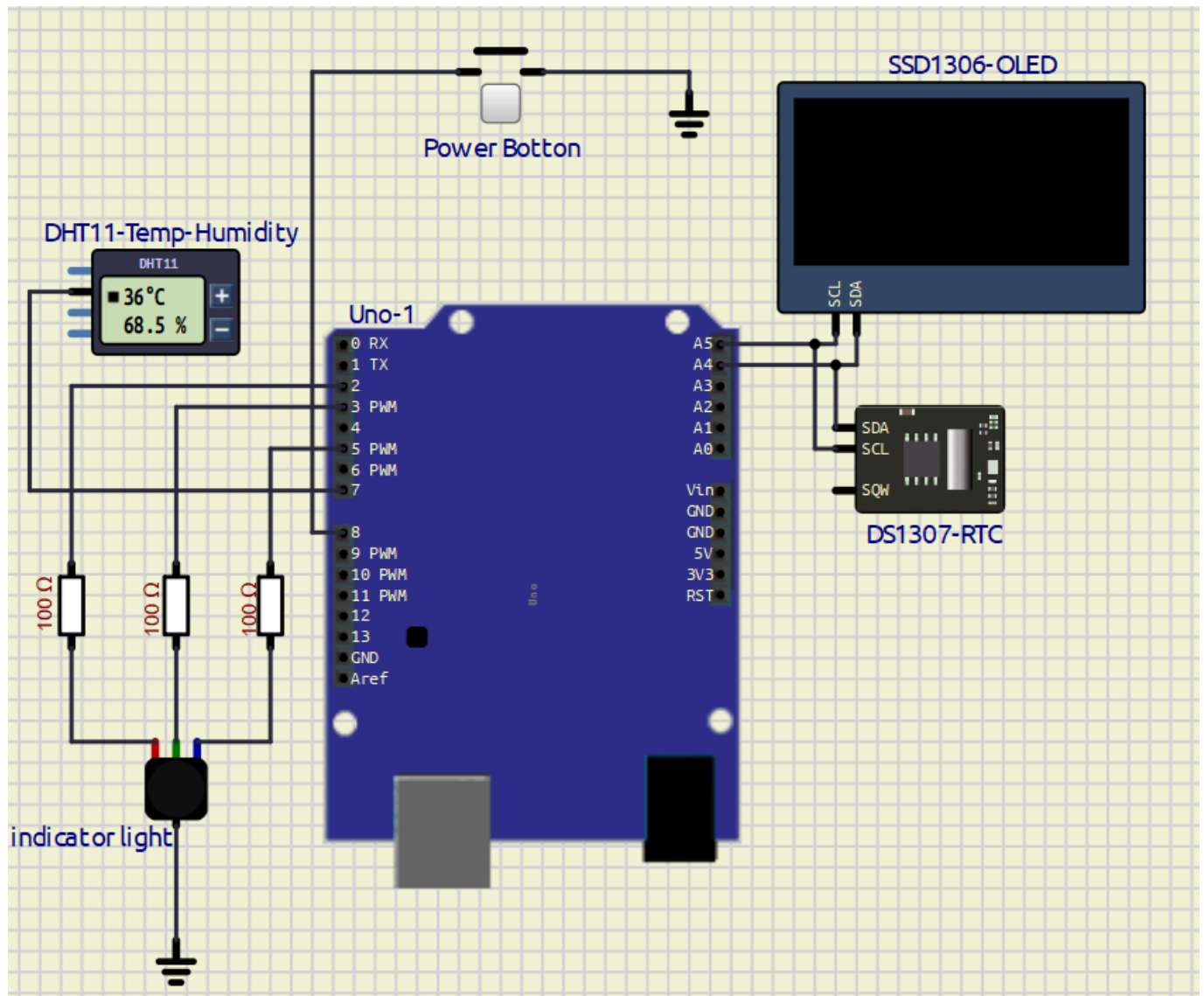
RGB LED CONNECTION

RGB Pin	Connection
Red Pin	D2 through 100Ω
Green Pin	D3 through 100Ω
Blue Pin	D5 through 100Ω
Common Cathode	GND

SOFTWARE USED

- Arduino IDE
- SimulIDE

CIRCUIT DIAGRAM



REQUIRED LIBRARIES

```
#include <Wire.h>
#include <RTClib.h>
#include <DHT.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
```

MAIN ARDUINO CODE

Code

```
#include <Wire.h>
#include <RTClib.h>

#include <DHT.h>
```

```
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64

Adafruit_SSD1306 display(
  SCREEN_WIDTH,
  SCREEN_HEIGHT,
  &Wire,
  -1
);

RTC_DS1307 rtc;

#define DHTPIN 7
#define DHTTYPE DHT11

DHT dht(DHTPIN, DHTTYPE);

#define RED_LED      2
#define GREEN_LED    3
#define BLUE_LED     5

#define POWER_BUTTON 8

bool systemState = false;

bool lastButtonState = HIGH;

void setup()
{
  pinMode(RED_LED, OUTPUT);

  pinMode(GREEN_LED, OUTPUT);

  pinMode(BLUE_LED, OUTPUT);

  pinMode(POWER_BUTTON, INPUT_PULLUP);

  Wire.begin();

  rtc.begin();

  dht.begin();

  // rtc.adjust(DateTime(F(__DATE__), F(__TIME__)));

  if (!display.begin(SSD1306_SWITCHCAPVCC, 0x3C))
  {
    while (1);
  }

  display.clearDisplay();
```



```
    display.display();
}

void loop()
{
    bool buttonState = digitalRead(POWER_BUTTON);

    if (
        buttonState == LOW &&
        lastButtonState == HIGH
    )
    {
        delay(50);

        systemState = !systemState;

        if (systemState == true)
        {
            display.clearDisplay();

            display.setTextSize(2);

            display.setTextColor(WHITE);

            display.setCursor(10, 25);

            display.println("POWER ON");

            display.display();

            delay(1000);
        }

        else
        {
            display.clearDisplay();

            display.setTextSize(2);

            display.setTextColor(WHITE);

            display.setCursor(10, 25);

            display.println("POWER OFF");

            display.display();

            delay(1000);

            display.clearDisplay();

            display.display();
        }
    }
}
```

```
}

lastButtonState = buttonState;

if (systemState == false)
{
    digitalWrite(RED_LED, LOW);

    digitalWrite(GREEN_LED, LOW);

    digitalWrite(BLUE_LED, LOW);

    return;
}

DateTime now = rtc.now();

float temperature = dht.readTemperature();

float humidity = dht.readHumidity();

String statusText;

if (temperature > 35)
{
    digitalWrite(RED_LED, HIGH);

    digitalWrite(GREEN_LED, LOW);

    digitalWrite(BLUE_LED, LOW);

    statusText = "EXTREME HOT";
}

else if (temperature < 20)
{
    digitalWrite(RED_LED, LOW);

    digitalWrite(GREEN_LED, LOW);

    digitalWrite(BLUE_LED, HIGH);

    statusText = "COLD";
}

else
{
    digitalWrite(RED_LED, LOW);

    digitalWrite(GREEN_LED, HIGH);

    digitalWrite(BLUE_LED, LOW);

    statusText = "NORMAL";
}
```

```
}

display.clearDisplay();

display.setTextSize(1);

display.setTextColor(WHITE);

display.setCursor(0, 0);

display.println("WEATHER SYSTEM");

display.setCursor(0, 15);

display.print("TIME : ");

print2digit(now.hour());

display.print(":");

print2digit(now.minute());

display.print(":");

print2digit(now.second());

display.setCursor(0, 30);

display.print("TEMP : ");

display.print(temperature);

display.println(" C");

display.setCursor(0, 42);

display.print("HUM : ");

display.print(humidity);

display.println(" %");

display.setCursor(0, 55);

display.print("STATUS:");

display.println(statusText);

display.display();

delay(50);
}
```

```
void print2digit(int number)
```

```
{  
  if (number < 10)  
  {  
    display.print("0");  
  }  
  
  display.print(number);  
}
```

WORKING PRINCIPLE

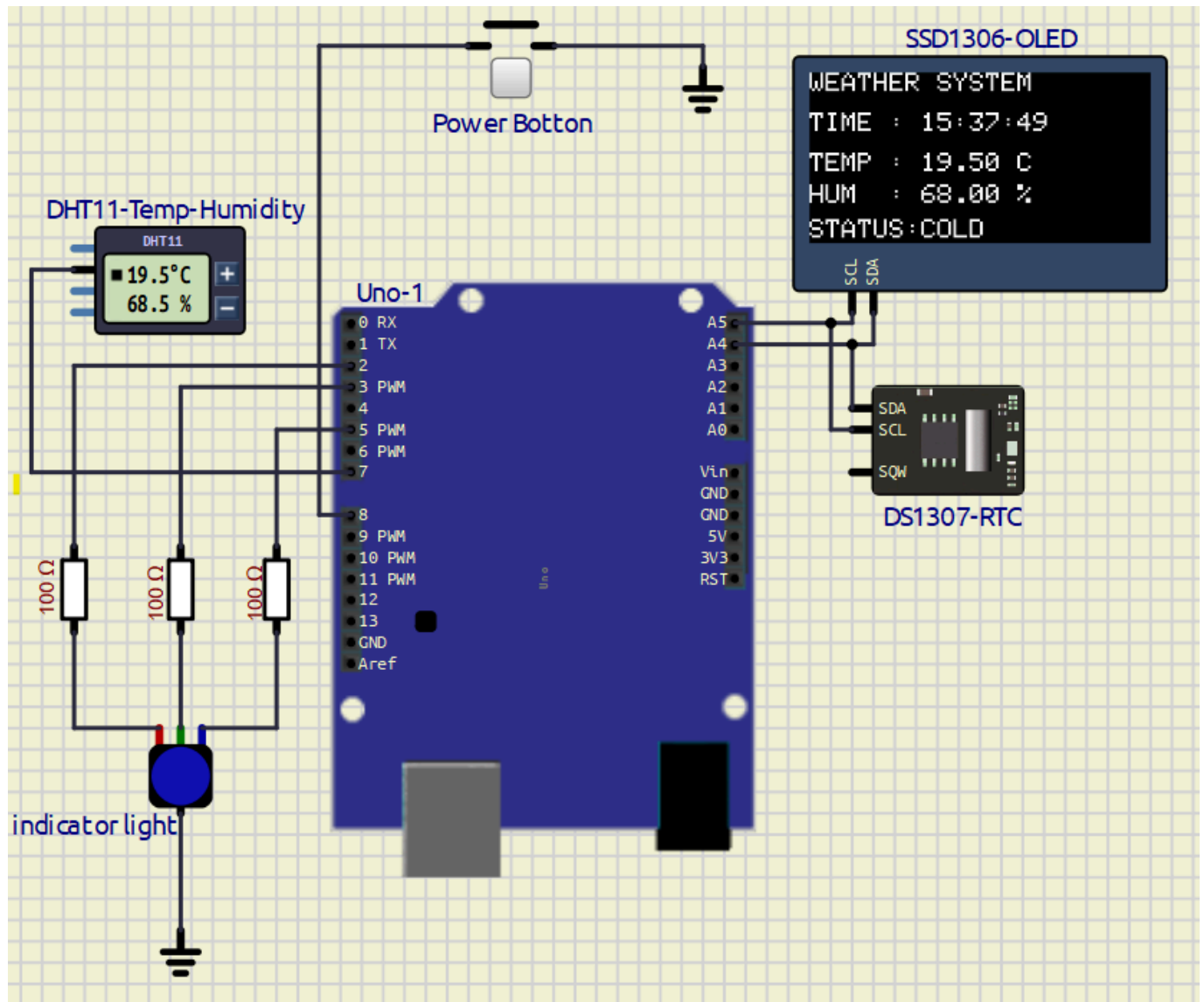
1. The system starts with a blank OLED screen.
2. Pressing the power button:
 - Turns the system ON
 - Displays "POWER ON"
3. DHT11 sensor continuously reads:
 - Temperature
 - Humidity
4. DS1307 RTC module provides:
 - Real-time clock
5. OLED displays:
 - Time
 - Temperature
 - Humidity
 - Weather Status
6. RGB LED indicates weather condition:
 - Red LED → Extreme Hot
 - Green LED → Normal Weather
 - Blue LED → Cold Weather
7. Pressing the power button again:
 - Displays "POWER OFF"
 - Turns OFF all LEDs
 - OLED returns to black screen

TEMPERATURE INDICATOR LOGIC

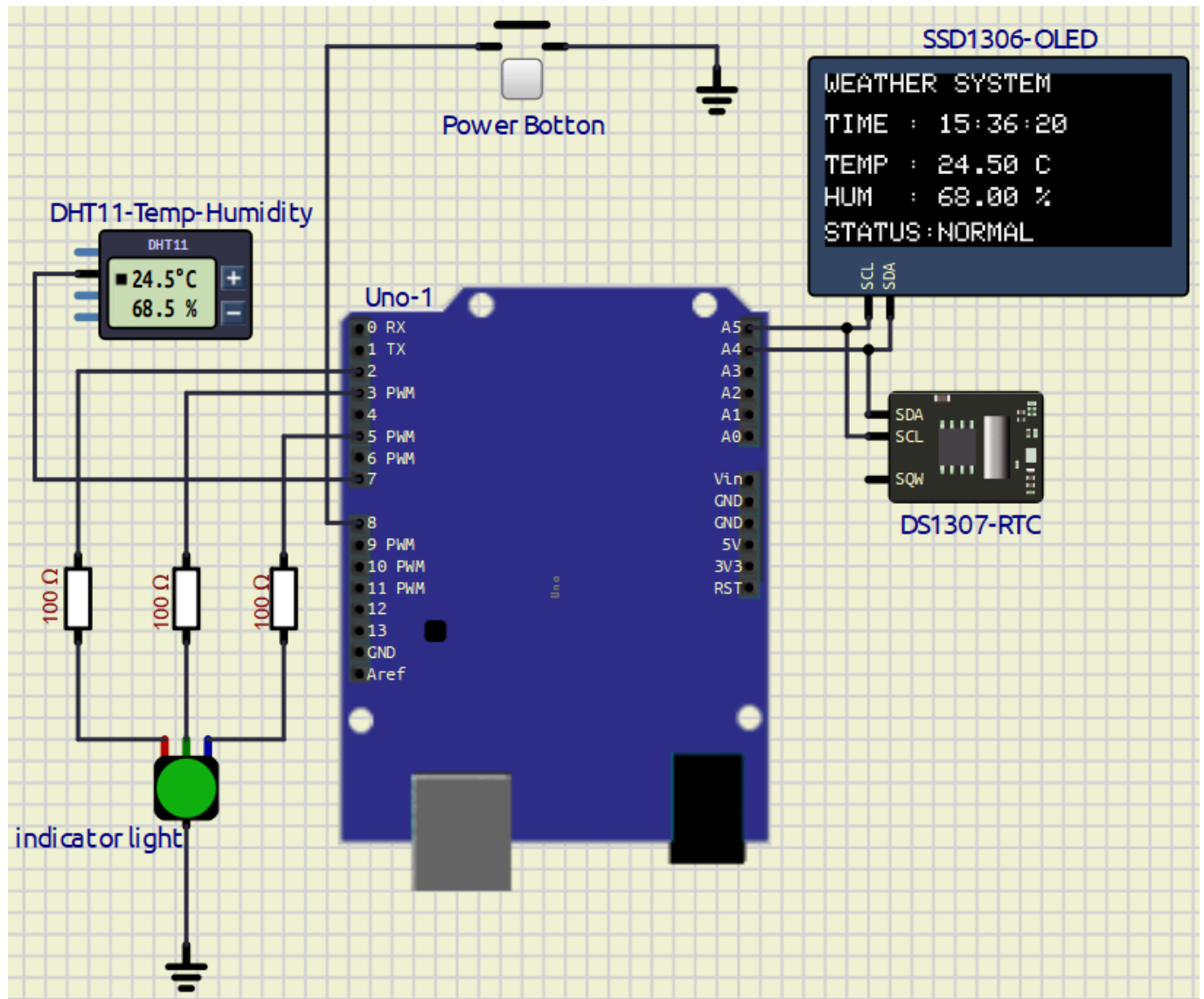
Temperature	LED Status	Weather
> 35°C	Red LED ON	Extreme Hot
20°C – 35°C	Green LED ON	Normal
< 20°C	Blue LED ON	Cold

OUTPUT

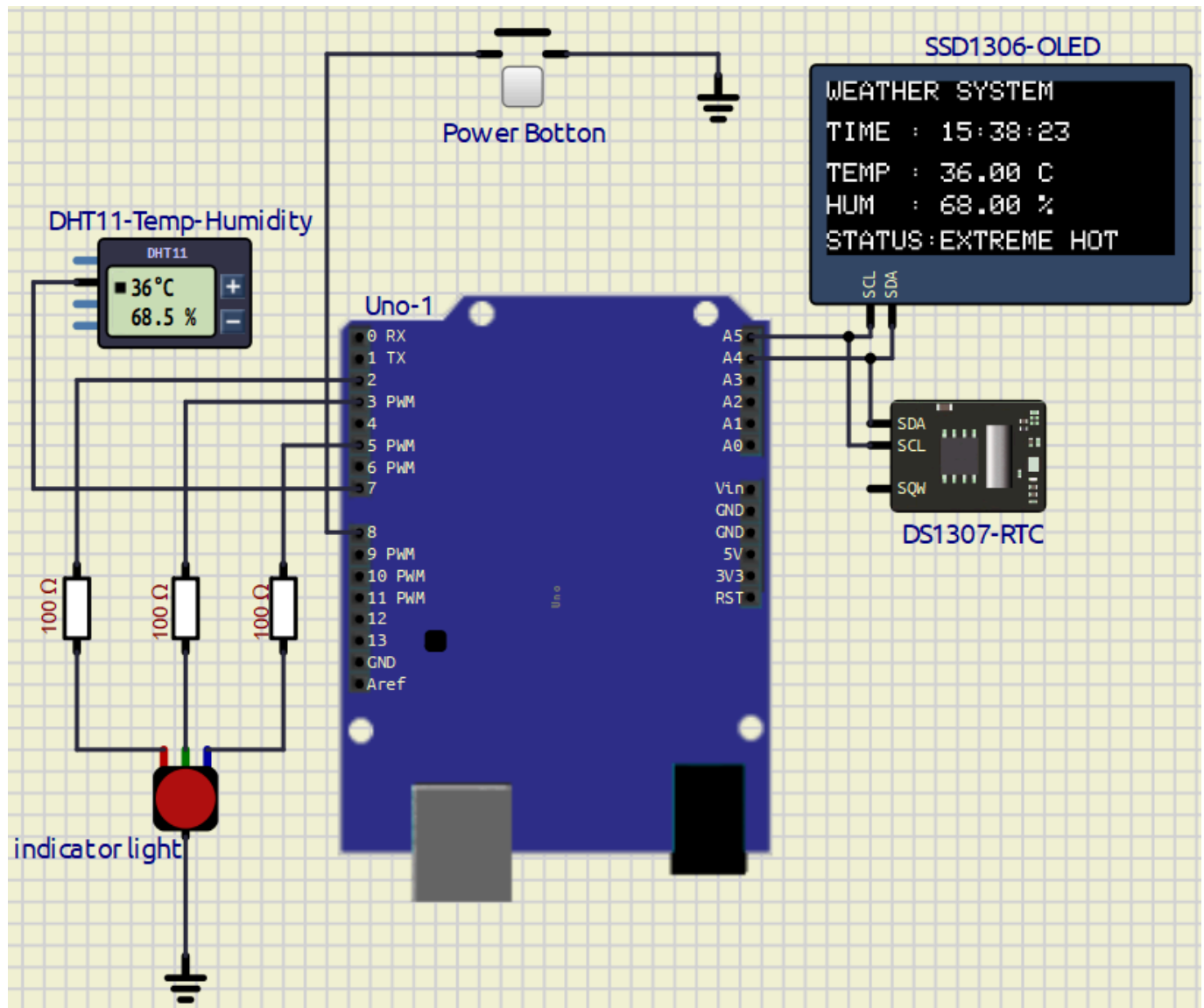
System - COLD (Temp < 20)



System - Normal (Temp > 20 & < 36)



System - Extream Hot (Temp > 35)



ADVANTAGES

- Real-time weather monitoring
- Simple embedded system design
- RGB visual indication
- RTC-based real-time display
- Easy ON/OFF control
- Low-cost implementation
- User-friendly interface

APPLICATIONS

- Smart weather stations
- Environmental monitoring
- Temperature warning systems
- Industrial monitoring
- Home automation systems
- Embedded systems learning

CONCLUSION

The Weather Forecasting System using Arduino UNO, DHT11 sensor, DS1307 RTC module, SSD1306 OLED display, RGB LED, and push-button control was successfully implemented in SimulIDE.

The system correctly monitors weather conditions, displays real-time environmental information, and visually indicates temperature conditions using RGB LEDs.

During this experiment, I learned:

- Sensor interfacing
- OLED interfacing
- RTC interfacing
- RGB LED control
- Push-button logic
- Embedded system design

REFERENCES

1. Arduino UNO Documentation
<https://docs.arduino.cc/hardware/uno-rev3/>
2. DHT11 Sensor Documentation
<https://components101.com/sensors/dht11-temperature-sensor>
3. DS1307 RTC Datasheet
<https://datasheets.maximintegrated.com/en/ds/DS1307.pdf>
4. Adafruit SSD1306 Library
https://github.com/adafruit/Adafruit_SSD1306
5. Adafruit GFX Library
<https://github.com/adafruit/Adafruit-GFX-Library>
6. RTCLib Documentation
<https://github.com/adafruit/RTCLib>

Ganesh Mahata 2022BB1197

EXPERIMENT– 2 Calculator

PROJECT NAME

Simple Calculator Using Arduino UNO, 4x4 Keypad and SSD1306 OLED Display

OBJECTIVE / PROBLEM STATEMENT

This project implements a simple calculator using Arduino UNO, a 4x4 keypad, and an SSD1306 OLED display. The calculator performs basic arithmetic operations such as addition, subtraction, multiplication, and division. The result is displayed on the OLED screen.

COMPONENTS USED

Component	Quantity
Arduino UNO	1
4x4 Keypad	1
SSD1306 OLED Display	1
Jumper Wires	Required
Breadboard	1

PIN CONNECTIONS

4x4 KEYPAD CONNECTIONS

Keypad Pin	Arduino Pin
R1	D1
R2	D2
R3	D3
R4	D4
C1	D5
C2	D6
C3	D7
C4	D8

Symbols mapped operators

Symbol	Operation
A	+
B	-
C	*
D	/
#	AC

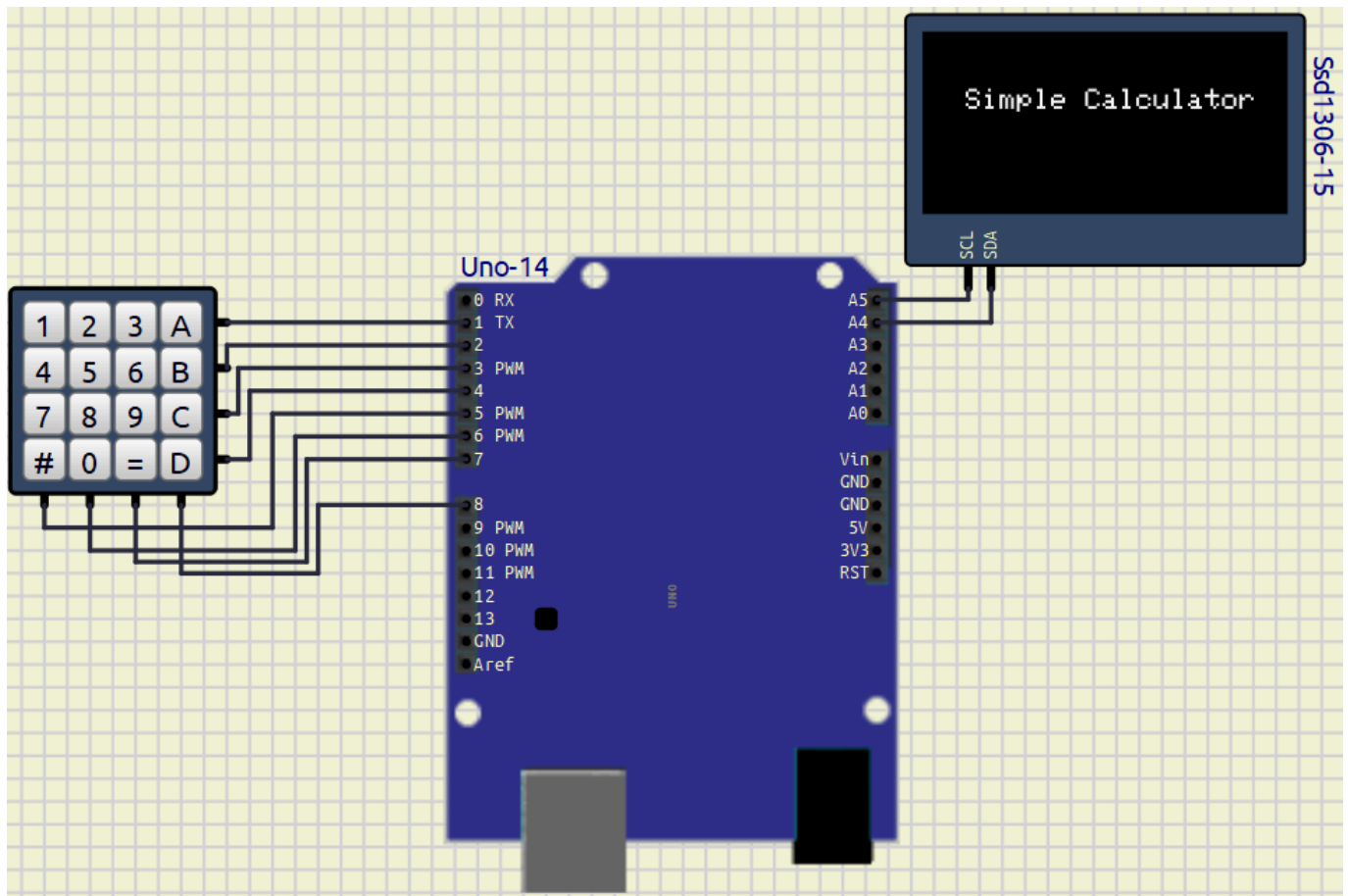
SSD1306 OLED DISPLAY CONNECTIONS

OLED Pin	Arduino Pin
VCC	5V
GND	GND
SDA	A4
SCL	A5

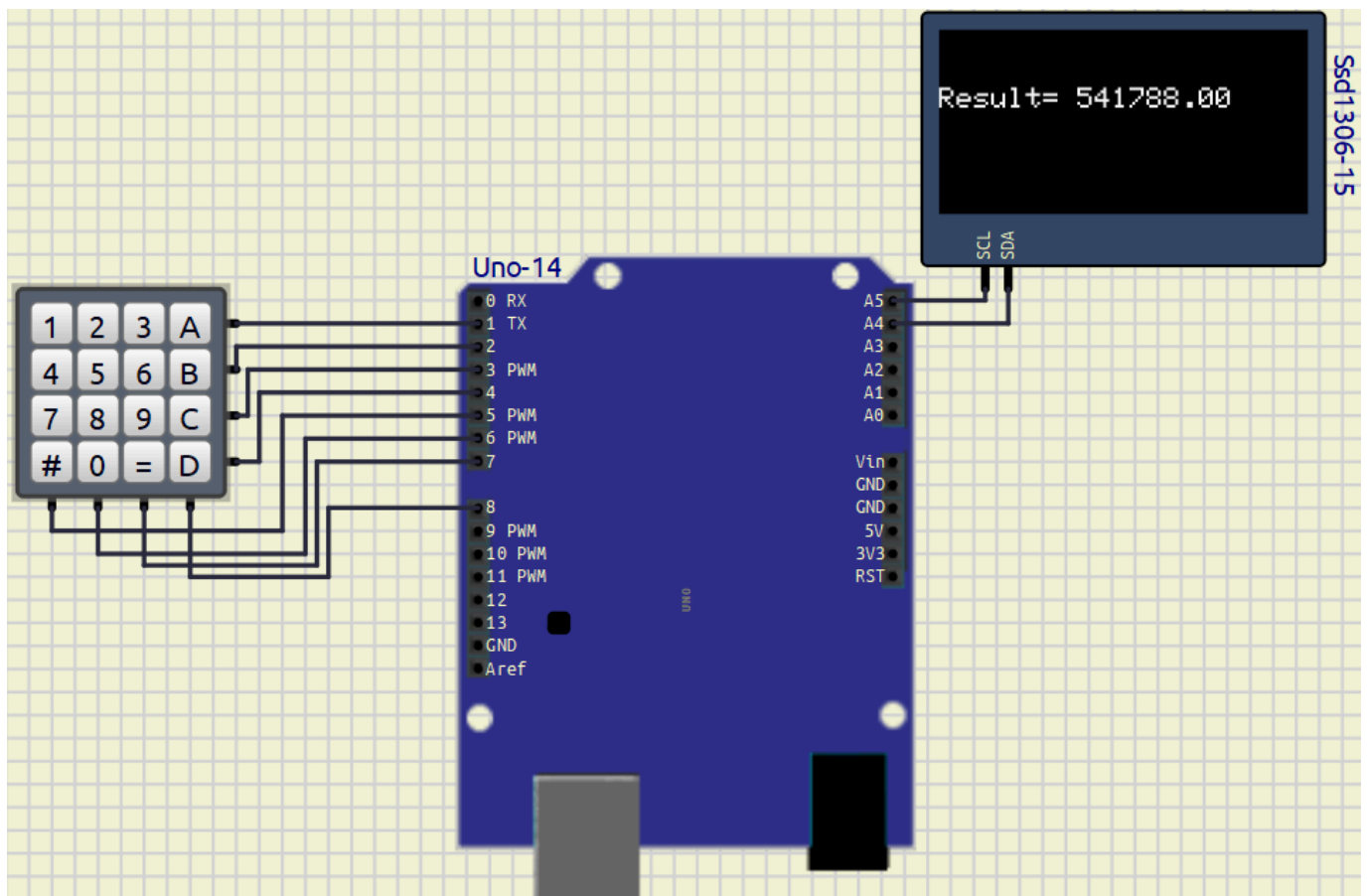
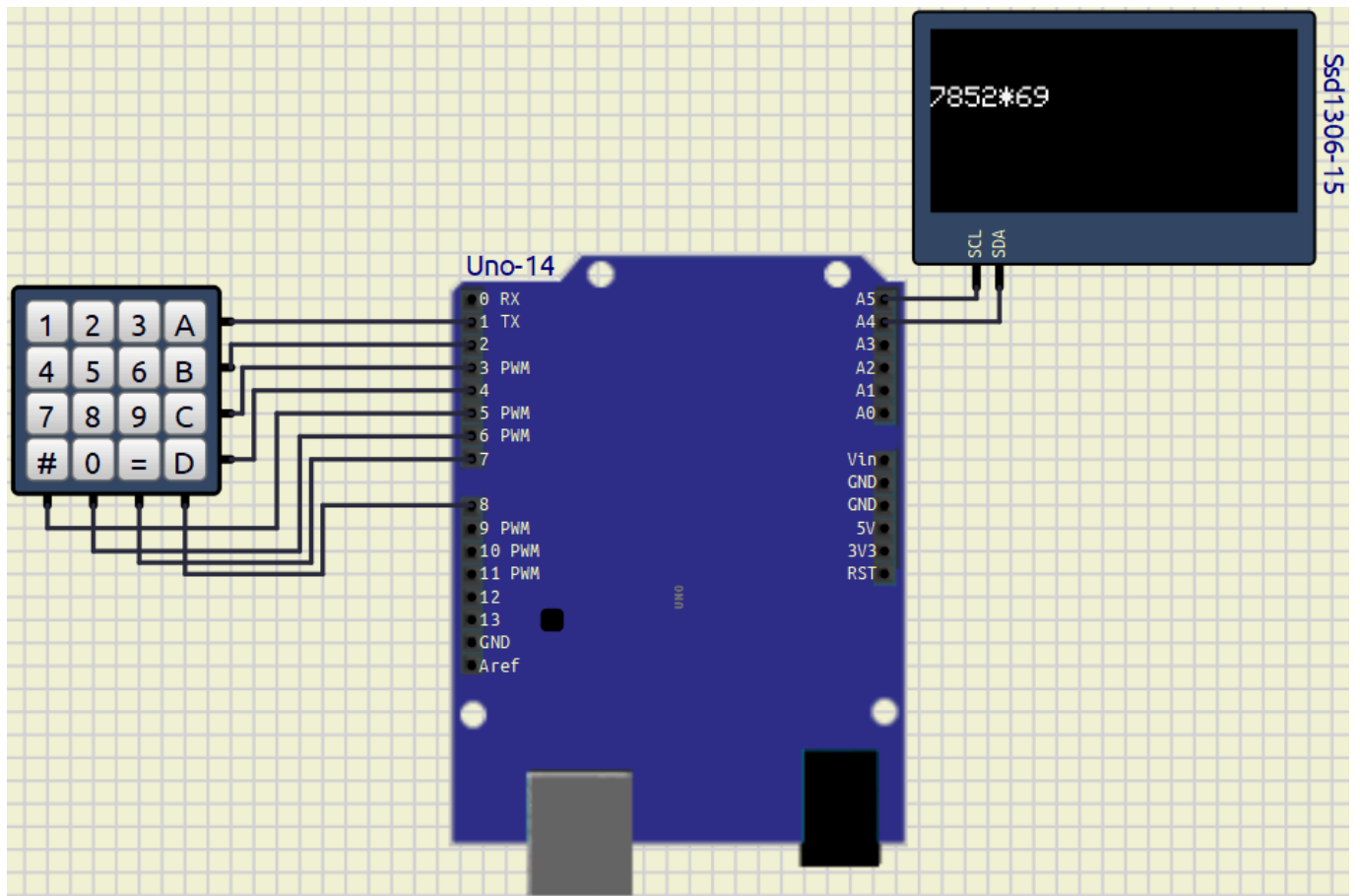
SOFTWARE USED

- Arduino IDE
 - SimulIDE
-

CIRCUIT DIAGRAM



Example:



REQUIRED LIBRARIES

```
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
#include <Keypad.h>
```

MAIN ARDUINO CODE

Code With Comments ---> [Click here](#)

```
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
#include <Keypad.h>

#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64

Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, -1);

const byte ROWS = 4;
const byte COLS = 4;

char keys[ROWS][COLS] = {
  {'1', '2', '3', '+'},
  {'4', '5', '6', '-'},
  {'7', '8', '9', '*'},
  {'#', '0', '=', '/'}
};

byte rowPins[ROWS] = {1, 2, 3, 4};
byte colPins[COLS] = {5, 6, 7, 8};

Keypad keypad = Keypad(makeKeymap(keys),
                        rowPins,
                        colPins,
                        ROWS,
                        COLS);

String num1 = "";
String num2 = "";

char op;

float result;

bool operatorPressed = false;

String expression = "";
```

```
void setup()
{
    display.begin(SSD1306_SWITCHCAPVCC, 0x3C);

    display.clearDisplay();

    display.setTextSize(1);

    display.setCursor(0, 0);
    display.println("Simple Calculator");

    display.setCursor(0, 20);
    display.println("Can Perform");

    display.setCursor(0, 40);
    display.println("(+, -, *, /)");

    display.display();

    delay(2000);

    display.clearDisplay();
    display.display();
}

void loop()
{
    char key = keypad.getKey();

    if(key)
    {
        // CLEAR ALL
        if(key == '#')
        {
            num1 = "";
            num2 = "";
            expression = "";

            operatorPressed = false;

            result = 0;

            display.clearDisplay();
            display.display();
        }

        // NUMBER INPUT
        else if(isDigit(key))
        {
            if(!operatorPressed)
            {
                num1 += key;
            }
            else
            {
                num2 += key;
            }
        }
    }
}
```

```
        {
            num2 += key;
        }

        expression += key;

        updateDisplay(expression);
    }

    // OPERATOR INPUT
    else if(key == '+' || key == '-' ||
            key == '*' || key == '/')
    {
        if(!operatorPressed && num1 != "")
        {
            op = key;

            operatorPressed = true;

            expression += key;

            updateDisplay(expression);
        }
    }

    // RESULT
    else if(key == '=')
    {
        float n1 = num1.toFloat();
        float n2 = num2.toFloat();

        switch(op)
        {
            case '+':
                result = n1 + n2;
                break;

            case '-':
                result = n1 - n2;
                break;

            case '*':
                result = n1 * n2;
                break;

            case '/':

                if(n2 != 0)
                {
                    result = n1 / n2;
                }
                else
                {
                    display.clearDisplay();
                }
            }
        }
    }
}
```

```
        display.setTextSize(2);

        display.setCursor(0, 20);

        display.println("ERROR");

        display.display();

        delay(2000);

        return;
    }

    break;
}

expression = "Result= " + String(result);

updateDisplay(expression);

num1 = String(result);

num2 = "";

operatorPressed = false;

expression = String(result);
    }
}

void updateDisplay(String text)
{
    display.clearDisplay();

    display.setTextSize(2);

    display.setCursor(0, 20);

    display.println(text);

    display.display();
}
```

WORKING PRINCIPLE

1. The 4x4 keypad is used to enter numbers and operators.
2. Arduino UNO continuously scans the keypad.
3. Entered values are stored as operands.
4. The selected operator determines the operation to be performed.

5. When '=' is pressed:
 - Arduino performs the calculation.
 - Result is displayed on the SSD1306 OLED display.
 6. '#' key clears all previous inputs.
 7. The process repeats continuously.
-

OUTPUT

Startup Screen

```
Simple Calculator
```

Example Calculation

```
5+3
```

```
Result= 8
```

FEATURES

- Performs Addition
 - Performs Subtraction
 - Performs Multiplication
 - Performs Division
 - OLED Display Output
 - Clear Function using '#'
 - Real-time Input Detection
 - Divide by Zero Error Handling
-

ADVANTAGES

- Simple and easy to implement
 - Low-cost embedded project
 - Real-time calculations
 - User-friendly interface
 - Useful for learning keypad interfacing
-

APPLICATIONS

- Basic calculator systems
- Embedded system projects

- Learning Arduino interfacing
 - Educational mini-projects
 - Digital input-output systems
-

CONCLUSION

The Simple Calculator using Arduino UNO, 4x4 Keypad, and SSD1306 OLED Display was successfully implemented and tested. The system successfully performs basic arithmetic operations and displays the result on the OLED screen. During the experiment I faces problem in mapping those keys and assign them their values. I overcome it by the open source resources given in web and arduino forum

REFERENCES

1. Arduino Keypad Library Documentation
<https://www.arduino.cc/reference/en/libraries/keypad/>
 2. SSD1306 OLED Display Library
https://github.com/adafruit/Adafruit_SSD1306
 3. Arduino UNO Official Documentation
<https://docs.arduino.cc/hardware/uno-rev3/>
 4. Adafruit GFX Graphics Library
<https://github.com/adafruit/Adafruit-GFX-Library>
 5. 4x4 Keypad Arduino Code, Pinout & Interfacing Complete Guide
<https://robosans.com/learn/embedded/arduino/4x4-keypad-arduino-code-pinout-interfacing-lcd/>
-

EXPERIMENT – 4 Alarm Clock Using RTC and OLED Display

PROJECT NAME

Real-Time Alarm Clock Using Arduino UNO, DS1307 RTC Module, SSD1306 OLED Display and Push Buttons

OBJECTIVE / PROBLEM STATEMENT

To interface a DS1307 Real-Time Clock (RTC) module with Arduino UNO using the I2C communication protocol and display the real-time clock data on an SSD1306 OLED display.

The system also implements an alarm-setting mechanism using push buttons. The user can set the alarm hour, minute, and second using dedicated push-button operations. When the current RTC time matches the preset alarm time, a buzzer sounds continuously until the time changes.

COMPONENTS USED

Component	Quantity
Arduino UNO	1
DS1307 RTC Module	1
SSD1306 OLED Display	1
Push Buttons	3
Buzzer	1
Jumper Wires	Required
Breadboard	1

PIN CONNECTIONS

PUSH BUTTON CONNECTIONS

Push Button	Arduino Pin	Function
P-adj	D1	Change Alarm Setting Mode
P+ve	D2	Increase Value
P-ve	D3	Decrease Value

BUZZER CONNECTION

Device	Arduino Pin
Buzzer (+)	D12
Buzzer (-)	GND

DS1307 RTC CONNECTIONS

RTC Pin	Arduino Pin
VCC	5V
GND	GND
SDA	A4
SCL	A5

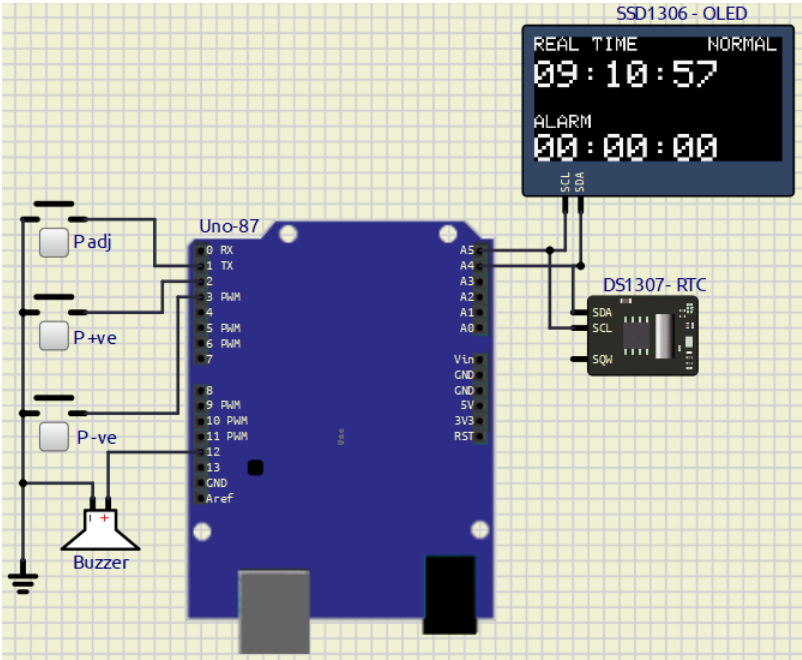
SSD1306 OLED DISPLAY CONNECTIONS

OLED Pin	Arduino Pin
VCC	5V
GND	GND
SDA	A4
SCL	A5

SOFTWARE USED

- Arduino IDE
 - Proteus / SimulIDE
-

CIRCUIT DIAGRAM



REQUIRED LIBRARIES

```
#include <Wire.h>
#include <RTClib.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
```

BUTTON OPERATION

P-adj Button Operation

Number of Press	Operation
1st Press	Activate Alarm Setting Mode
2nd Press	Set Hour
3rd Press	Set Minute
4th Press	Set Second
5th Press	Exit Alarm Setting Mode

P+ve Button

- Increases selected value

P-ve Button

- Decreases selected value

MAIN ARDUINO CODE

Code With Comments ---> [Click Here](#)

```
#include <Wire.h>
#include <RTClib.h>

#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64

Adafruit_SSD1306 display(SCREEN_WIDTH,
                          SCREEN_HEIGHT,
                          &Wire,
                          -1);

RTC_DS1307 rtc;

// BUTTON PINS

#define BTN_ADJ 1
#define BTN_INC 2
#define BTN_DEC 3

// BUZZER PIN

#define BUZZER 12

// ALARM VARIABLES

int alarmHour = 0;
int alarmMinute = 0;
int alarmSecond = 0;

// MODE VARIABLE

/*
mode = 0 -> Normal Mode
mode = 1 -> Set Hour
mode = 2 -> Set Minute
mode = 3 -> Set Second
*/

int mode = 0;
```

```
bool alarmTriggered = false;

// BUTTON STATES

bool lastAdjState = HIGH;
bool lastIncState = HIGH;
bool lastDecState = HIGH;

void setup()
{
    pinMode(BTN_ADJ, INPUT_PULLUP);
    pinMode(BTN_INC, INPUT_PULLUP);
    pinMode(BTN_DEC, INPUT_PULLUP);

    pinMode(BUZZER, OUTPUT);

    digitalWrite(BUZZER, LOW);

    Wire.begin();

    rtc.begin();

    // Uncomment first time only
    // rtc.adjust(DateTime(F(__DATE__), F(__TIME__)));

    if (!display.begin(SSD1306_SWITCHCAPVCC, 0x3C))
    {
        while (1);
    }

    display.clearDisplay();
    display.display();

    delay(1000);
}

void loop()
{
    DateTime now = rtc.now();

    handleButtons();

    checkAlarm(now);

    displayData(now);

    delay(150);
}
```

```
void handleButtons()
{
    bool adjState = digitalRead(BTN_ADJ);
    bool incState = digitalRead(BTN_INC);
    bool decState = digitalRead(BTN_DEC);

    // P-ADJ BUTTON

    if (adjState == LOW && lastAdjState == HIGH)
    {
        delay(200);

        mode++;

        if (mode > 4)
        {
            mode = 0;
        }
    }

    // P+ BUTTON

    if (incState == LOW && lastIncState == HIGH)
    {
        delay(200);

        if (mode == 1)
        {
            alarmHour++;

            if (alarmHour > 23)
                alarmHour = 0;
        }

        else if (mode == 2)
        {
            alarmMinute++;

            if (alarmMinute > 59)
                alarmMinute = 0;
        }

        else if (mode == 3)
        {
            alarmSecond++;

            if (alarmSecond > 59)
```



```
        alarmSecond = 0;
    }
}

// P- BUTTON

if (decState == LOW && lastDecState == HIGH)
{
    delay(200);

    if (mode == 1)
    {
        alarmHour--;

        if (alarmHour < 0)
            alarmHour = 23;
    }

    else if (mode == 2)
    {
        alarmMinute--;

        if (alarmMinute < 0)
            alarmMinute = 59;
    }

    else if (mode == 3)
    {
        alarmSecond--;

        if (alarmSecond < 0)
            alarmSecond = 59;
    }
}

lastAdjState = adjState;
lastIncState = incState;
lastDecState = decState;
}

// CHECK ALARM

void checkAlarm(DateTime now)
{
    // Trigger Alarm

    if (
        now.hour() == alarmHour &&
```

```
    now.minute() == alarmMinute &&
    now.second() == alarmSecond
)
{
    alarmTriggered = true;
}

// Continuous Buzzer

if (alarmTriggered)
{
    tone(BUZZER, 1000);
}

// Stop Alarm When Time Changes

if (
    now.hour() != alarmHour ||
    now.minute() != alarmMinute
)
{
    alarmTriggered = false;

    noTone(BUZZER);
}
}

// DISPLAY FUNCTION

void displayData(DateTime now)
{
    display.clearDisplay();

    display.setTextSize(1);
    display.setTextColor(WHITE);

    // REAL TIME

    display.setCursor(0, 0);
    display.println("REAL TIME");

    display.setTextSize(2);

    display.setCursor(0, 12);

    print2digit(now.hour());
    display.print(":");
```

```
print2digit(now.minute());
display.print(":");

print2digit(now.second());


// ALARM TIME

display.setTextSize(1);

display.setCursor(0, 40);
display.println("ALARM");

display.setTextSize(2);

display.setCursor(0, 50);

print2digit(alarmHour);
display.print(":");

print2digit(alarmMinute);
display.print(":");

print2digit(alarmSecond);


// MODE DISPLAY

display.setTextSize(1);

display.setCursor(90, 0);

if (mode == 0)
{
    display.println("NORMAL");
}

else if (mode == 1)
{
    display.println("SET HR");
}

else if (mode == 2)
{
    display.println("SET MIN");
}

else if (mode == 3)
{
    display.println("SET SEC");
}
```

```
display.display();  
}  
  
void print2digit(int number)  
{  
    if (number < 10)  
    {  
        display.print("0");  
    }  
  
    display.print(number);  
}
```

WORKING PRINCIPLE

1. DS1307 RTC module continuously maintains the current real-time clock.
2. Arduino UNO reads the RTC time through I2C communication.
3. SSD1306 OLED display shows:
 - Current Time
 - Alarm Time
 - Current Setting Mode
4. P-adj button changes the alarm setting mode.
5. P+ and P- buttons increase or decrease the selected time unit.
6. Alarm hour, minute, and second are stored in memory.
7. When RTC time matches the alarm time:
 - Arduino activates the buzzer.
 - Buzzer rings continuously.
8. Alarm automatically stops when the time changes.

OUTPUT

OLED Display

Real-Time Clock Display

```
REAL TIME  
12:45:30  
  
ALARM  
06:00:00
```

Alarm Trigger

Current Time = Alarm Time
Buzzer Activated

FEATURES

- Real-time clock using DS1307 RTC
- Alarm setting functionality
- OLED display output
- Push-button controlled interface
- Continuous buzzer alarm
- I2C communication
- Hour, minute, and second adjustment
- Low-cost embedded system project

ADVANTAGES

- Accurate real-time tracking
- Simple alarm configuration
- Low power consumption
- Easy hardware interfacing
- Compact embedded system
- User-friendly operation

APPLICATIONS

- Digital alarm clocks
- Embedded timing systems
- Reminder systems
- Home automation projects
- Educational Arduino projects
- RTC interfacing experiments

CONCLUSION

The Alarm Clock System using Arduino UNO, DS1307 RTC Module, SSD1306 OLED Display, and push buttons was successfully implemented and tested. The system correctly displays the real-time clock, allows users to set alarm parameters, and activates the buzzer when the alarm time matches the RTC time.

During the experiment, I learned about:

- I2C communication protocol
 - RTC interfacing
 - OLED display handling
 - Push-button interfacing
 - Alarm logic implementation in embedded systems
-

REFERENCES

1. Arduino UNO Official Documentation
<https://docs.arduino.cc/hardware/uno-rev3/>
 2. DS1307 RTC Module Documentation
<https://datasheets.maximintegrated.com/en/ds/DS1307.pdf>
 3. RTCLib Library Documentation
<https://github.com/adafruit/RTCLib>
 4. Adafruit SSD1306 OLED Library
https://github.com/adafruit/Adafruit_SSD1306
 5. Adafruit GFX Graphics Library
<https://github.com/adafruit/Adafruit-GFX-Library>
 6. I2C Communication Protocol Tutorial
<https://www.arduino.cc/en/reference/wire>
-

Ganesh Mahata - 2022BB11977

EXPERIMENT – 5 Alarm Based Motor ON/OFF System

PROJECT NAME

Alarm Based Motor ON/OFF System Using Arduino UNO, DS1307 RTC Module and SSD1306 OLED Display

OBJECTIVE / PROBLEM STATEMENT

To design and implement an automatic motor ON/OFF control system using Arduino UNO. The system detects water supply using a switch sensor and controls a pump indicator LED based on water availability.

The DS1307 RTC module is used for real-time tracking and timing operations. The SSD1306 OLED display continuously displays real time, water supply status, and pump status.

The system also checks whether water supply continues for more than one hour.

COMPONENTS USED

Component	Quantity
Arduino UNO	1
DS1307 RTC Module	1
SSD1306 OLED Display	1
Push Button / Water Sensor Switch	1
LED (Pump Indicator)	1
Jumper Wires	Required
Breadboard	1

PIN CONNECTIONS

WATER SENSOR CONNECTION

Device	Arduino Pin
Water Supply Switch	D2

PUMP LED CONNECTION

Device	Arduino Pin
Pump LED	D3

DS1307 RTC CONNECTIONS

RTC Pin	Arduino Pin
VCC	5V
GND	GND
SDA	A4
SCL	A5

SSD1306 OLED DISPLAY CONNECTIONS

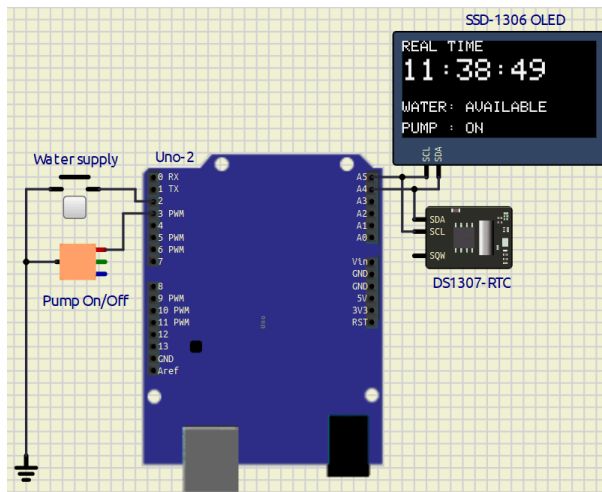
OLED Pin	Arduino Pin
VCC	5V
GND	GND
SDA	A4
SCL	A5

SOFTWARE USED

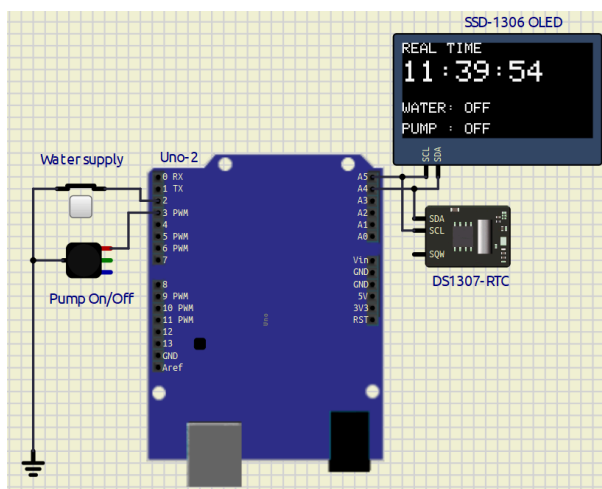
- Arduino IDE
 - SimulIDE / Proteus
-

CIRCUIT DIAGRAM

1. When Water supply available



2. When water supply is off



REQUIRED LIBRARIES

```
#include <Wire.h>
#include <RTClib.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
```

MAIN ARDUINO CODE

Code With Comment ----> [Click Here](#)

```
/*
=====
Alarm Based Motor ON/OFF System using RTC
=====
*/
```

```
#include <Wire.h>
#include <RTClib.h>

#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

// OLED SETTINGS

#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64

Adafruit_SSD1306 display(SCREEN_WIDTH,
                          SCREEN_HEIGHT,
                          &Wire,
                          -1);

// RTC OBJECT

RTC_DS1307 rtc;

// PIN DEFINITIONS

#define WATER_SENSOR 2
#define PUMP_LED 3

// VARIABLES

bool pumpState = false;

bool timerStarted = false;

// Time when water supply started

DateTime waterStartTime;

// SETUP

void setup()
{
  pinMode(WATER_SENSOR, INPUT_PULLUP);

  pinMode(PUMP_LED, OUTPUT);

  digitalWrite(PUMP_LED, LOW);
```

```
Wire.begin();

rtc.begin();

// Uncomment first time only
// rtc.adjust(DateTime(F(__DATE__), F(__TIME__)));

// OLED START

if (!display.begin(SSD1306_SWITCHCAPVCC, 0x3C))
{
    while (1);
}

display.clearDisplay();
display.display();

delay(1000);
}

// MAIN LOOP

void loop()
{
    DateTime now = rtc.now();

    bool waterAvailable = digitalRead(WATER_SENSOR);

    // WATER AVAILABLE

    if (waterAvailable == LOW)
    {
        // Turn Pump ON

        digitalWrite(PUMP_LED, HIGH);

        pumpState = true;

        // Start Timer Only Once

        if (!timerStarted)
        {
            waterStartTime = now;

            timerStarted = true;
        }
    }
}
```

```
// Calculate Time Difference

TimeSpan elapsed = now - waterStartTime;

// Check 1 Hour Condition

if (elapsed.TotalSeconds() >= 3600)
{
    // Additional logic can be added here
}

// WATER NOT AVAILABLE

else
{
    timerStarted = false;

    // Wait 2 Seconds

    delay(2000);

    digitalWrite(PUMP_LED, LOW);

    pumpState = false;
}

// UPDATE OLED DISPLAY

displayData(now, waterAvailable);

delay(200);
}

// OLED DISPLAY FUNCTION

void displayData(DateTime now, bool waterAvailable)
{
    display.clearDisplay();

    display.setTextSize(1);

    display.setTextColor(WHITE);
```

```
// REAL TIME

display.setCursor(0, 0);

display.println("REAL TIME");

display.setTextSize(2);

display.setCursor(0, 12);

print2digit(now.hour());

display.print(":");

print2digit(now.minute());

display.print(":");

print2digit(now.second());


// WATER STATUS

display.setTextSize(1);

display.setCursor(0, 40);

display.print("WATER: ");

if (waterAvailable == LOW)
{
    display.println("AVAILABLE");
}
else
{
    display.println("OFF");
}


// PUMP STATUS

display.setCursor(0, 54);

display.print("PUMP : ");

if (pumpState)
{
    display.println("ON");
}
else
{
```

```
    display.println("OFF");
}

display.display();
}

// PRINT 2 DIGIT NUMBER

void print2digit(int number)
{
    if (number < 10)
    {
        display.print("0");
    }

    display.print(number);
}
```

WORKING PRINCIPLE

1. Water sensor continuously checks water supply.
2. If water is available:
 - Pump LED turns ON.
 - RTC timer starts.
3. If water supply continues for more than 1 hour:
 - Timer condition becomes true.
4. If water supply stops before 1 hour:
 - System waits 2 seconds.
 - Pump LED turns OFF.
5. OLED continuously displays:
 - Real Time
 - Water Supply Status
 - Pump Status

OUTPUT

OLED Display

```
REAL TIME
12:45:30

WATER: AVAILABLE
```

PUMP : ON

Water Supply OFF

REAL TIME
12:46:10

WATER: OFF

PUMP : OFF

FEATURES

- Automatic pump control
- Real-time clock display
- OLED status monitoring
- Water detection system
- 1-hour timing condition
- I2C communication
- Simple embedded automation system

ADVANTAGES

- Easy to implement
- Low-cost project
- Real-time monitoring
- Automatic control system
- Useful for water management systems

APPLICATIONS

- Water tank automation
- Motor protection systems
- Smart irrigation systems
- Industrial timing systems
- Embedded automation projects

CONCLUSION

The Alarm Based Motor ON/OFF System using Arduino UNO, DS1307 RTC module, and SSD1306 OLED display was successfully implemented and tested. The system correctly detects water supply conditions, controls pump operation, and displays real-time status information on the OLED display.

During this experiment, I learned:

- RTC interfacing
- OLED interfacing
- I2C communication
- Digital input/output control
- Embedded automation logic

REFERENCES

1. Arduino UNO Official Documentation
<https://docs.arduino.cc/hardware/uno-rev3/>
2. DS1307 RTC Datasheet
<https://datasheets.maximintegrated.com/en/ds/DS1307.pdf>
3. RTCLib Library
<https://github.com/adafruit/RTCLib>
4. Adafruit SSD1306 Library
https://github.com/adafruit/Adafruit_SSD1306
5. Adafruit GFX Library
<https://github.com/adafruit/Adafruit-GFX-Library>
6. Arduino Wire Library Documentation
<https://www.arduino.cc/en/reference/wire>

EXPERIMENT – 4 Using the I2C Bus

PROJECT NAME

Temperature Data Logger Using Arduino UNO, DHT11 Sensor, DS1307 RTC Module and SSD1306 OLED Display

PROBLEM STATEMENT

To design and implement a real-time temperature monitoring and recording system using Arduino UNO. The system continuously reads temperature using a DHT11 sensor and records temperature values every 15 minutes.

The DS1307 RTC module provides accurate real-time timestamps, while the SSD1306 OLED display shows live temperature and time information. The recorded temperature data is stored in Arduino EEPROM memory and transferred to a laptop through USB serial communication.

COMPONENTS USED

Component	Quantity
Arduino UNO	1
DHT11 Temperature Sensor	1
DS1307 RTC Module	1
SSD1306 OLED Display	1
Jumper Wires	Required
Breadboard	1
USB Cable	1

PIN CONNECTIONS

DHT11 SENSOR CONNECTIONS

DHT11 Pin	Arduino Pin
DATA	D2
VCC	5V
GND	GND

DS1307 RTC CONNECTIONS

RTC Pin	Arduino Pin
SDA	A4
SCL	A5
VCC	5V
GND	GND

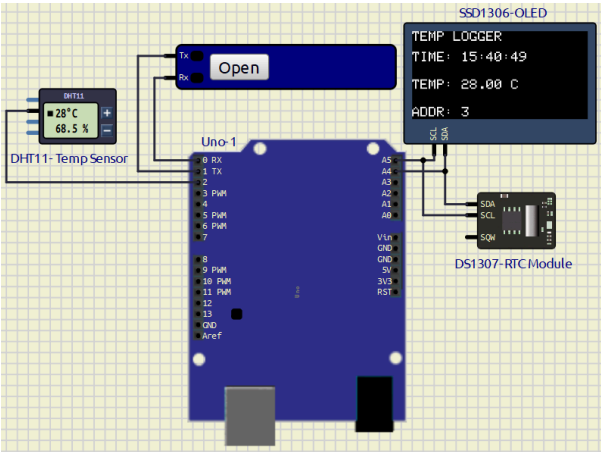
SSD1306 OLED CONNECTIONS

OLED Pin	Arduino Pin
SDA	A4
SCL	A5
VCC	5V
GND	GND

SOFTWARE USED

- Arduino IDE
- SimulIDE

Circuit Diagram



WORKING PRINCIPLE

1. DHT11 continuously measures temperature.
2. DS1307 RTC provides real-time clock information.
3. Arduino UNO reads:
 - Temperature
 - Current time

4. OLED display shows:
 - Current Time
 - Current Temperature
 - EEPROM Address
 5. Every 15 minutes:
 - Temperature value is stored into EEPROM.
 - Data is transferred to laptop through USB serial communication.
 6. Serial monitor displays recorded temperature logs.
-

LIBRARIES USED

```
#include <Wire.h>
#include <RTClib.h>
#include <EEPROM.h>
#include <DHT.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
```

MAIN ARDUINO CODE

Complete Code With Comments----->[Click here](#)

```
#include <Wire.h>
#include <RTClib.h>
#include <EEPROM.h>
#include <DHT.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64

Adafruit_SSD1306 display(
  SCREEN_WIDTH,
  SCREEN_HEIGHT,
  &Wire,
  -1
);

RTC_DS1307 rtc;

#define DHTPIN 2
#define DHTTYPE DHT11

DHT dht(DHTPIN, DHTTYPE);

int eepromAddress = 0;
```

```
int lastRecordedMinute = -1;

void setup()
{
  Serial.begin(9600);

  Wire.begin();

  rtc.begin();

  dht.begin();

  // rtc.adjust(DateTime(F(__DATE__), F(__TIME__)));

  if (!display.begin(SSD1306_SWITCHCAPVCC, 0x3C))
  {
    while (1);
  }

  display.clearDisplay();
  display.display();

  Serial.println("TEMP LOGGER STARTED");
}

void loop()
{
  DateTime now = rtc.now();

  float temperature = dht.readTemperature();

  displayData(now, temperature);

  if (
    now.minute() % 15 == 0 &&
    now.minute() != lastRecordedMinute
  )
  {
    EEPROM.write(eepromAddress++, now.hour());

    EEPROM.write(eepromAddress++, now.minute());

    EEPROM.write(eepromAddress++, (int)temperature);

    Serial.print(now.hour());

    Serial.print(":");

    print2digitSerial(now.minute());

    Serial.print(":");

    print2digitSerial(now.second());
```

```
        Serial.print(",");

        Serial.print(temperature);

        Serial.println(" C");

        lastRecordedMinute = now.minute();
    }

    delay(1000);
}

void displayData(DateTime now, float temperature)
{
    display.clearDisplay();

    display.setTextSize(1);

    display.setTextColor(WHITE);

    display.setCursor(0, 0);
    display.println("TEMP LOGGER");

    display.setCursor(0, 15);

    display.print("TIME: ");

    print2digit(now.hour());

    display.print(":");

    print2digit(now.minute());

    display.print(":");

    print2digit(now.second());

    display.setCursor(0, 35);

    display.print("TEMP: ");

    display.print(temperature);

    display.println(" C");

    display.setCursor(0, 55);

    display.print("ADDR: ");

    display.print(eepromAddress);

    display.display();
}
```

```
void print2digit(int number)
{
    if (number < 10)
    {
        display.print("0");
    }

    display.print(number);
}

void print2digitSerial(int number)
{
    if (number < 10)
    {
        Serial.print("0");
    }

    Serial.print(number);
}
```

OLED OUTPUT

TEMP LOGGER

TIME: 12:15:00

TEMP: 25 C

ADDR: 3

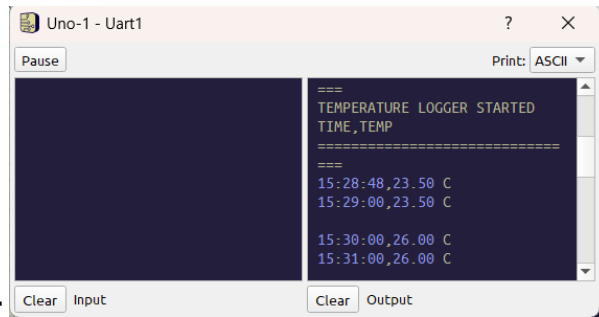
SERIAL OUTPUT

12:00:00,25 C

12:01:00,26 C

12:02:00,27 C

Instead of record data each 15min , I record it each 1 min onwards. so this is how its recorded and



displays in serial monitor

FEATURES

- Real-time temperature monitoring
 - Temperature recording every 15 minutes
 - OLED display output
 - RTC-based time stamping
 - EEPROM data storage
 - USB serial data transfer
 - Laptop data recording support
-

ADVANTAGES

- No SD card required
 - Low-cost implementation
 - Easy USB communication
 - Real-time monitoring
 - Permanent EEPROM storage
 - Simple embedded system design
-

APPLICATIONS

- Weather monitoring systems
 - Laboratory temperature logging
 - Industrial monitoring systems
 - Greenhouse monitoring
 - Environmental monitoring
 - Embedded system experiments
-

CONCLUSION

The Temperature Data Logger System using Arduino UNO, DHT11 sensor, DS1307 RTC module, and SSD1306 OLED display was successfully implemented and tested. The system accurately records temperature every 15

minutes and transfers the data to a laptop through USB serial communication.

During this experiment, I learned:

- RTC interfacing
- OLED interfacing
- EEPROM memory handling
- Serial communication
- Temperature sensing
- Data logging techniques in embedded systems

REFERENCES

1. Arduino UNO Official Documentation
<https://docs.arduino.cc/hardware/uno-rev3/>
 2. DHT11 Sensor Documentation
<https://components101.com/sensors/dht11-temperature-sensor>
 3. DS1307 RTC Datasheet
<https://datasheets.maximintegrated.com/en/ds/DS1307.pdf>
 4. RTCLib Library
<https://github.com/adafruit/RTCLib>
 5. Adafruit SSD1306 Library
https://github.com/adafruit/Adafruit_SSD1306
 6. EEPROM Library Documentation
<https://www.arduino.cc/en/Reference/EEPROM>
-

Ganesh Mahata 2022BB1197